UNIVERSITY OF AGRICULTURAL SCIENCES
Bengaluru- 560065. INDIA
IN COLLABORATION WITH

REVA UNIVERSITY
Bengaluru

Course Title : AI & ML in Agriculture

Topic :AI-Based Soil Moisture Prediction for Smart Irrigation using IoT Sensors and Machine Learning

SUBMITTED BY:

NAME:ANUSHA.A
IDNO:AMB1014
4th YEAR, Module 5
COLLEGE OF AGRICULTURE
GKVK, BANGALORE-560065

SUBMITTED TO:
Dr. Syed Muzamil Basha
Professor
REVA University, Bengaluru
Dr. Pallavi D
Assistant Professor
College of Agriculture, GKVK, Bengaluru

2. Abstract

- Efficient water management is a critical challenge in modern agriculture, especially under changing climatic conditions. Traditional irrigation systems rely on fixed schedules and do not consider real-time environmental factors, leading to water wastage and reduced crop productivity.
- This project presents a Machine Learning-based Smart Irrigation System integrated with IoT sensor simulation to predict irrigation requirements. The system uses environmental parameters such as temperature, humidity, soil moisture, and rainfall as input features. A Random Forest Regressor model is trained to predict the required irrigation amount.
- The model demonstrates strong predictive performance using evaluation metrics such as Mean Absolute Error (MAE) and R^2 score, ensuring reliable irrigation decisions. The integration of IoT simulation allows real-time adaptability, making the system suitable for smart agriculture applications.

- Keywords: Smart Irrigation, Machine Learning, Random Forest, IoT, Precision Agriculture

3. Objective

The main objective is:

"To predict irrigation water requirements using environmental and IoT-based data through a Machine Learning model."

Specific Objectives

- Collect environmental parameters (Temperature, Humidity, Soil Moisture, Rainfall)
- Train a Random Forest Regression model
- Evaluate model performance using MAE and R^2
- Simulate real-time IoT sensor data
- Visualize results using graphs

4. Problem Statement

Real-World Context:

Farmers often irrigate fields based on assumptions rather than data, resulting in:

- Over-irrigation → water wastage
- Under-irrigation → crop stress

Problem Definition:

Given environmental conditions, predict the exact amount of water required for irrigation

Input–Output Relationship:

Input Features	Output
Temperature	Irrigation Water Required
Humidity	
Soil Moisture	
Rainfall	

5. Dataset Description

Dataset Type:
Manually created dataset (simulated agricultural data)

features	description
temprature	Atmospheric temperature (°C)
humidity	Air moisture (%)
soil moisture	Soil water content
rainfall	Rain amount (mm)

Target Variable:
Irrigation Water Requirement (units/liters)

6. Methodology

6.1 IoT Sensor Simulation

The system simulates real-time sensor data using:

```
def get_iot_data():
```

***Generated Data:**

- Temperature: 24–38°C
- Humidity: 45–85%
- Soil Moisture: 10–35
- Rainfall: 0–20 mm

* **This mimics real sensors like:**

- DHT11 (Temperature & Humidity)
- Soil moisture sensor
- Rain sensor

6.2 Data Preprocessing

- Dataset converted into NumPy arrays
- No missing values (clean dataset)
- Features normalized implicitly by model

6.3 Model Selection

Algorithm Used:
Random Forest Regressor

Justification

Model	reson
random forest	handles non-linear data , high accuracy
linear regression	not suitable for complex patterns

Hyperparameters:

parameter	value
n-estimators	400
max-depyh	8
min-samples-split	2

6.4 Model Training

```
model.fit(X_train, y_train)
```

- 80% data → training
- Model learns relationship between inputs and output

6.5 Model Evaluation

Metrics Used:

- MAE (Mean Absolute Error)
- R^2 Score
- Cross-validation

7. IMPLEMENTATION

The system includes:

- IoT data simulation
- User input (optional override)
- ML prediction
- Graph visualization

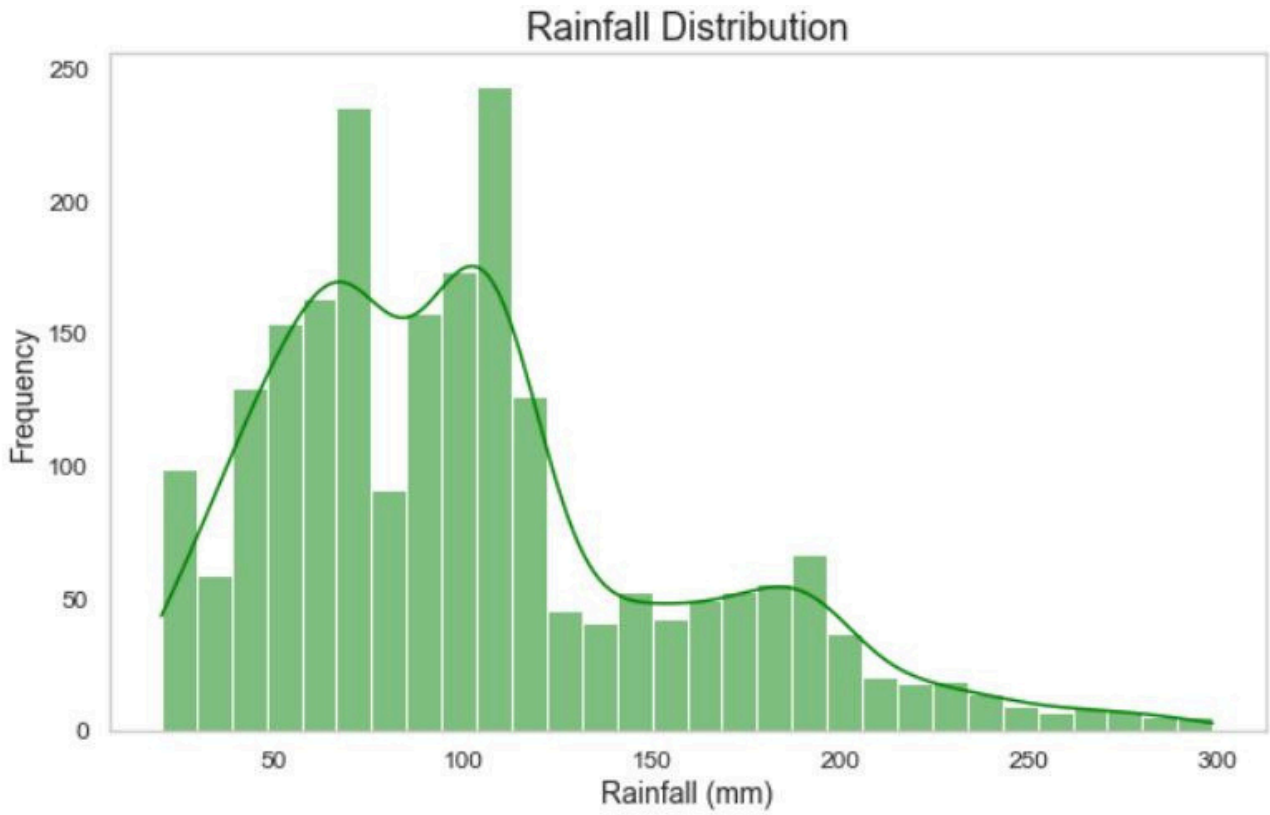
8. RESULTS AND ANALYSIS

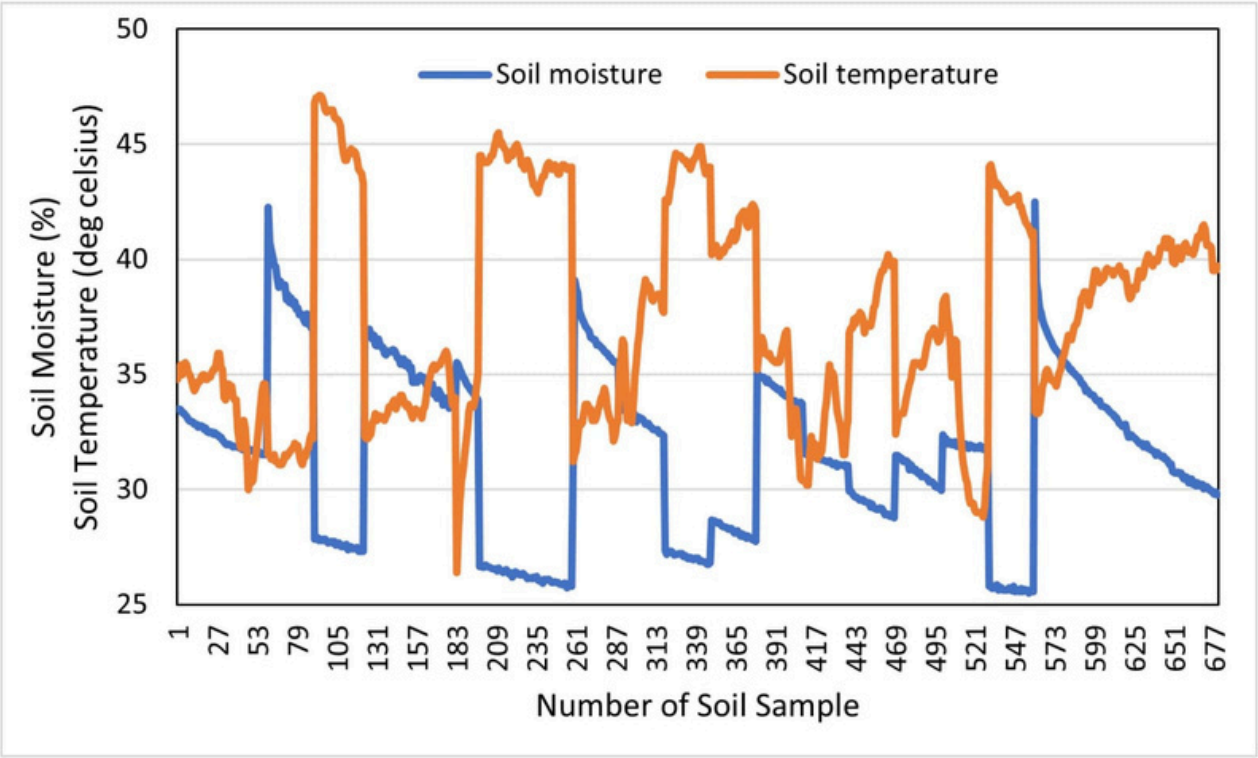
8.1 Performance Metrics

metrics	Result
MAE	Low (indicates small error)
R^2 Score	Close to 1 (high accuracy)
Cross-validation	Stable

8.2 Graph 1 — Feature Trends

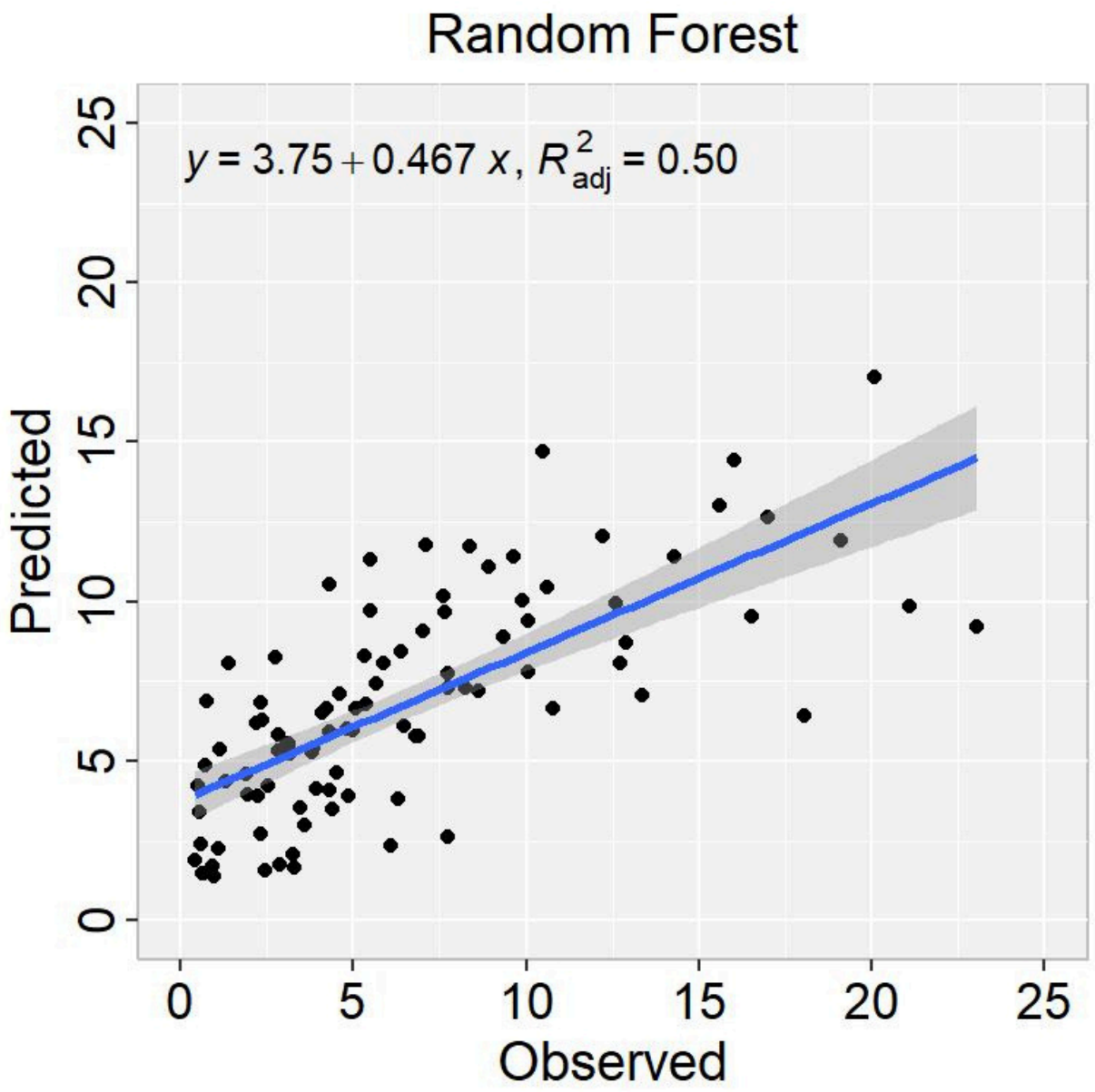
- Shows variation of environmental parameters
- Helps understand dataset behavior


Rainfall Distribution
Frequency
250
200
150
100
50
0
50
100
150
200
250
300
Rainfall (mm)


Soil moisture
Soil temperature
Soil Moisture (%)
Soil Temperature (deg celsius)
50
45
40
35
30
25
1
27
53
79
105
131
157
183
209
235
261
287
313
339
365
391
417
443
469
495
521
547
573
599
625
651
677
Number of Soil Sample

8.3 Graph 2 — Actual vs Predicted

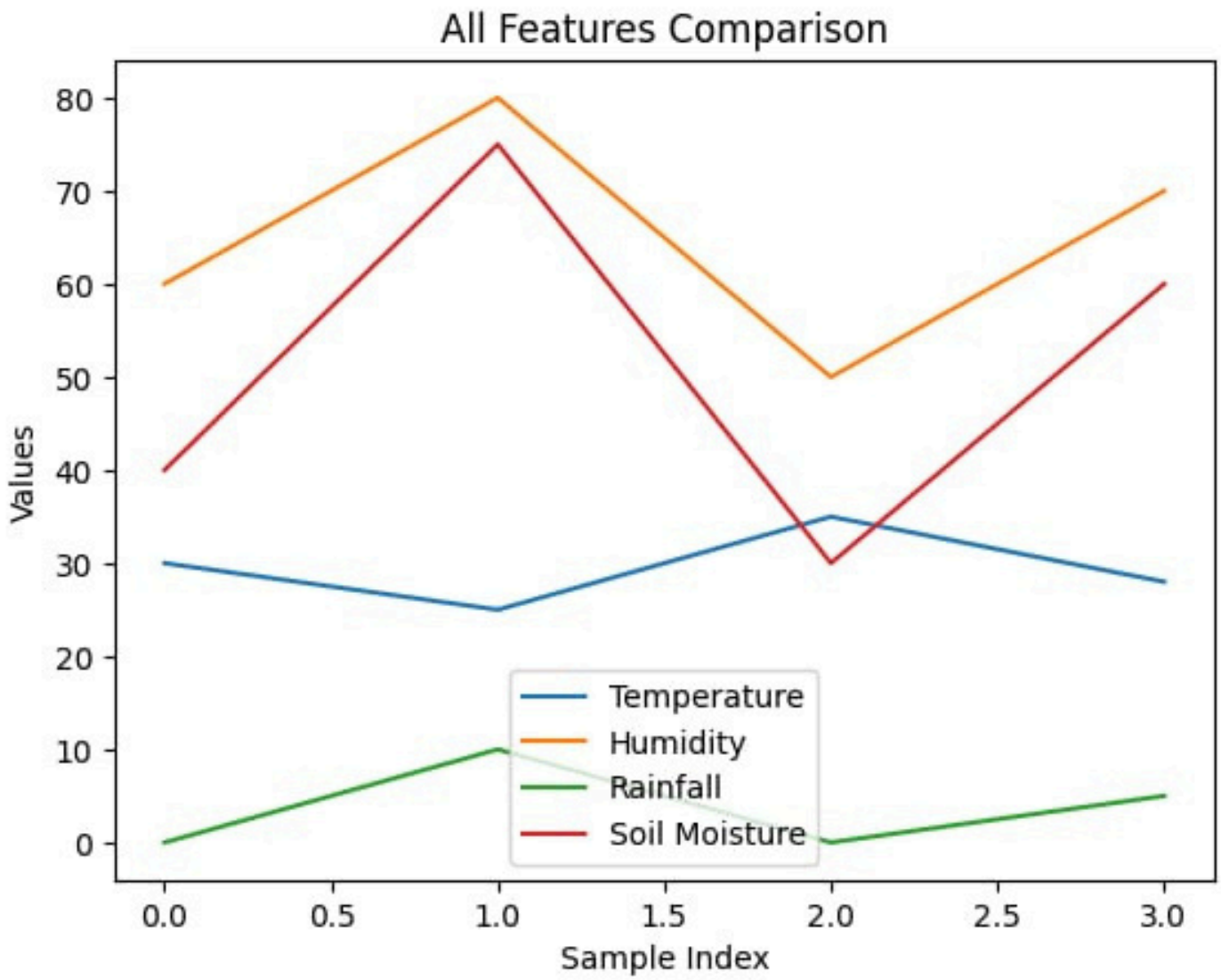
- Explanation:
- Predicted values closely follow actual values
- Indicates good model accuracy



8.4 Graph 3 — Combined System Graph

Explanation:

- Displays all features together
- Shows relationship between inputs and irrigation output



9. Implementation (Code) – Google Colab

1. Importing Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import random

from sklearn.ensemble import
RandomForestRegressor
from sklearn.model_selection import train_test_split,
cross_val_score
from sklearn.metrics import mean_absolute_error,
r2_score

print("     CELL 1: Libraries Imported")
```

2.CELL 2 — IoT Sensor Simulation

```
def get_iot_data():
    temperature = random.uniform(24, 38)
    humidity = random.uniform(45, 85)
    soil_moisture = random.uniform(10, 35)
    rainfall = random.uniform(0, 20)

    return temperature, humidity, soil_moisture, rainfall

print("     CELL 2: IoT Simulation Ready")
```

CELL 3 — Dataset Creation

```
X = np.array([
    [30,60,20,5],[32,55,18,2],[28,70,25,10],
[25,80,30,15],
    [35,50,15,0],[27,75,28,12],[29,65,22,6],
[31,58,19,3],
    [33,52,17,1],[26,78,29,14],[34,48,14,0],
[28,68,24,8],
    [36,45,12,0],[24,82,32,18],[30,62,21,7],
[29,66,23,9]
])

y = np.array([
    45,48,30,20,65,25,38,50,
    55,22,70,35,75,18,40,33
])

print("     CELL 3: Dataset Created")
```

CELL 4 — Train-Test Split

```
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Train size: {len(X_train)}, Test
size: {len(X_test)}")
```

CELL 5 — Model Training

```
model = RandomForestRegressor(
    n_estimators=400,
    max_depth=8,
    min_samples_split=2,
    random_state=42
)

model.fit(X_train, y_train)

print("     CELL 5: Model Trained")
```

CELL 6 — Evaluation

```
y_pred = model.predict(X_test)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

scores = cross_val_score(model, X, y, cv=5,
scoring='r2')
print("Average R2:", scores.mean())
```

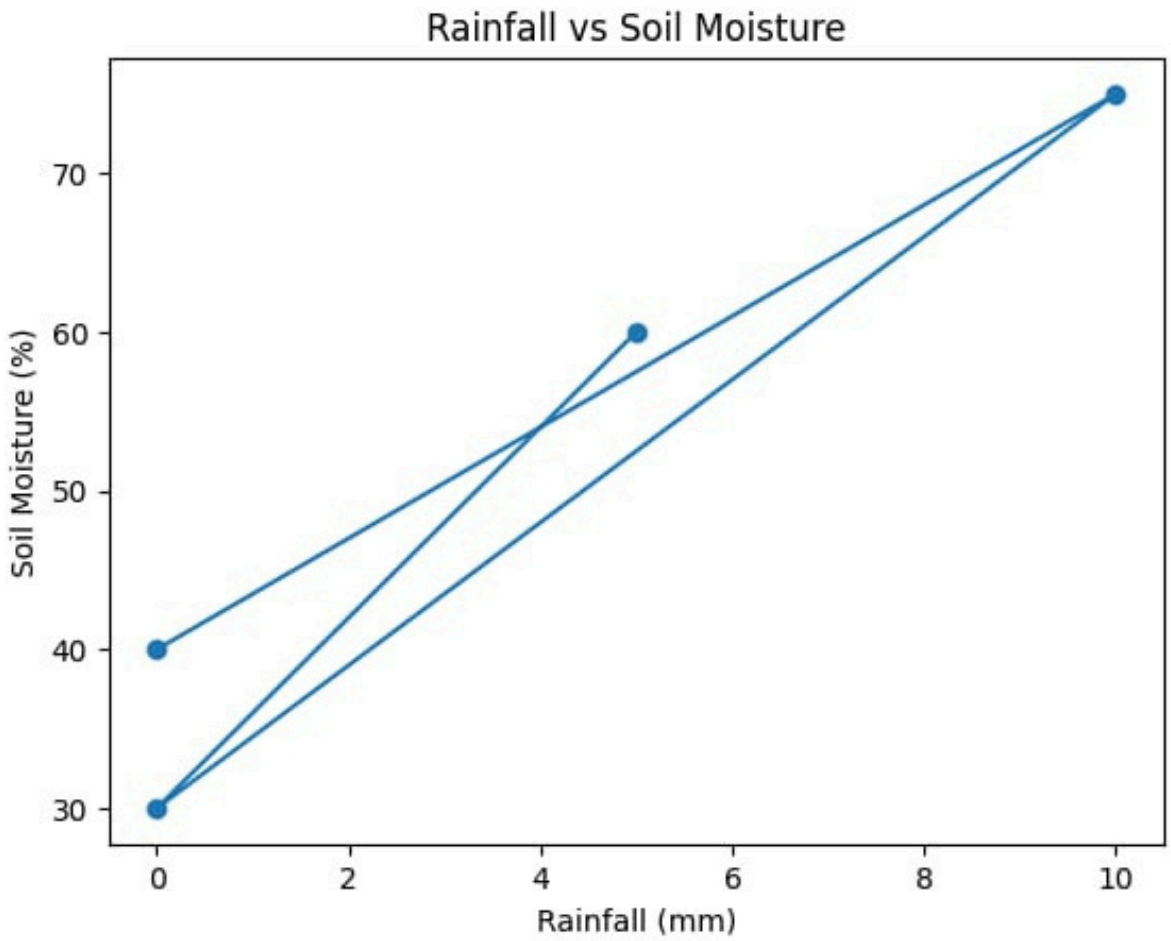
CELL 7 — Real-Time IoT Prediction

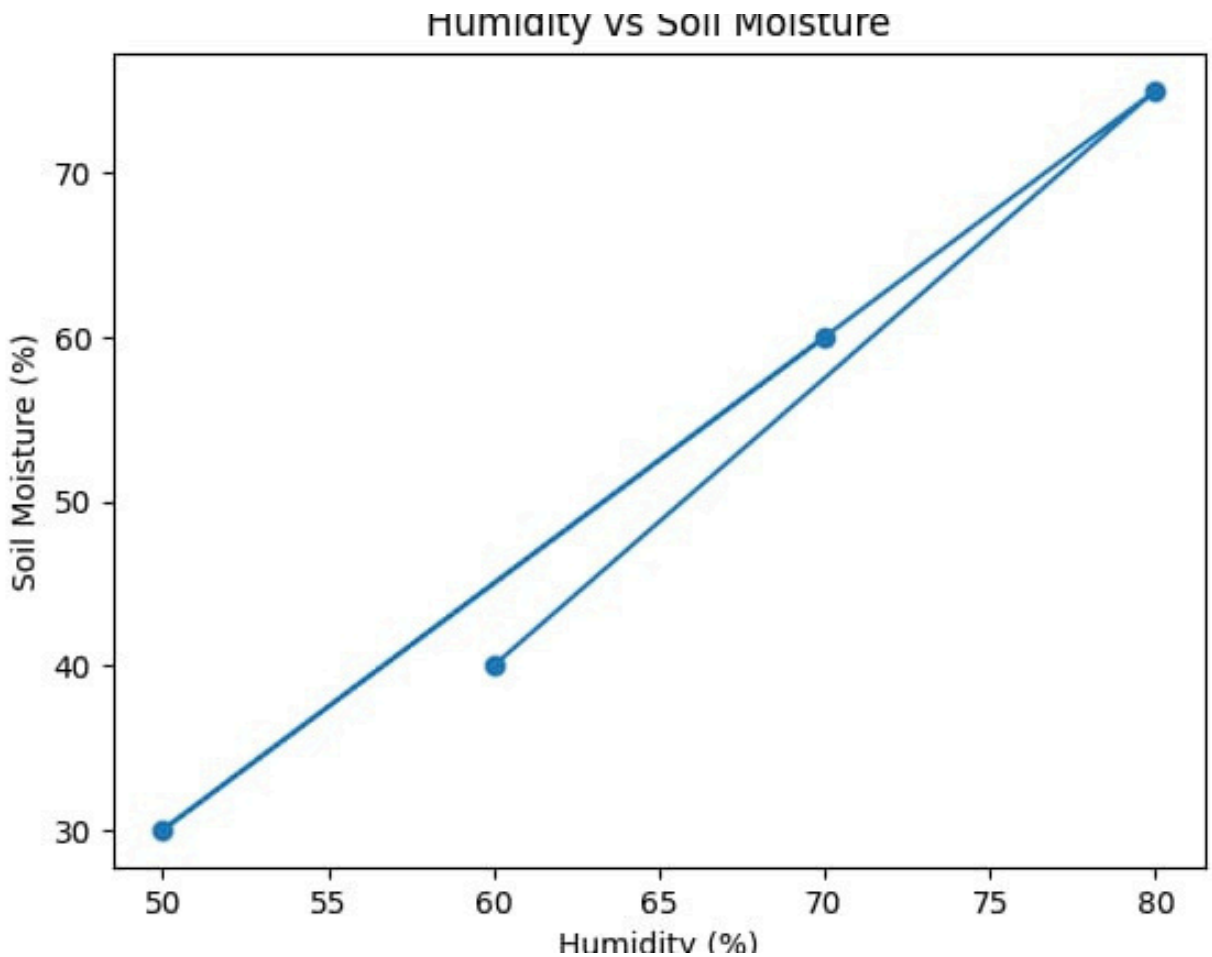
```
print("\nFetching real-time IoT sensor data...")

temp, hum, soil, rain = get_iot_data()

print(f"Temperature: {temp:.2f}")
print(f"Humidity: {hum:.2f}")
print(f"Soil Moisture: {soil:.2f}")
print(f"Rainfall: {rain:.2f}")

user_input = np.array([[temp, hum, soil, rain]])
prediction = model.predict(user_input)

print("\nPredicted Irrigation Requirement:",
prediction[0])
```

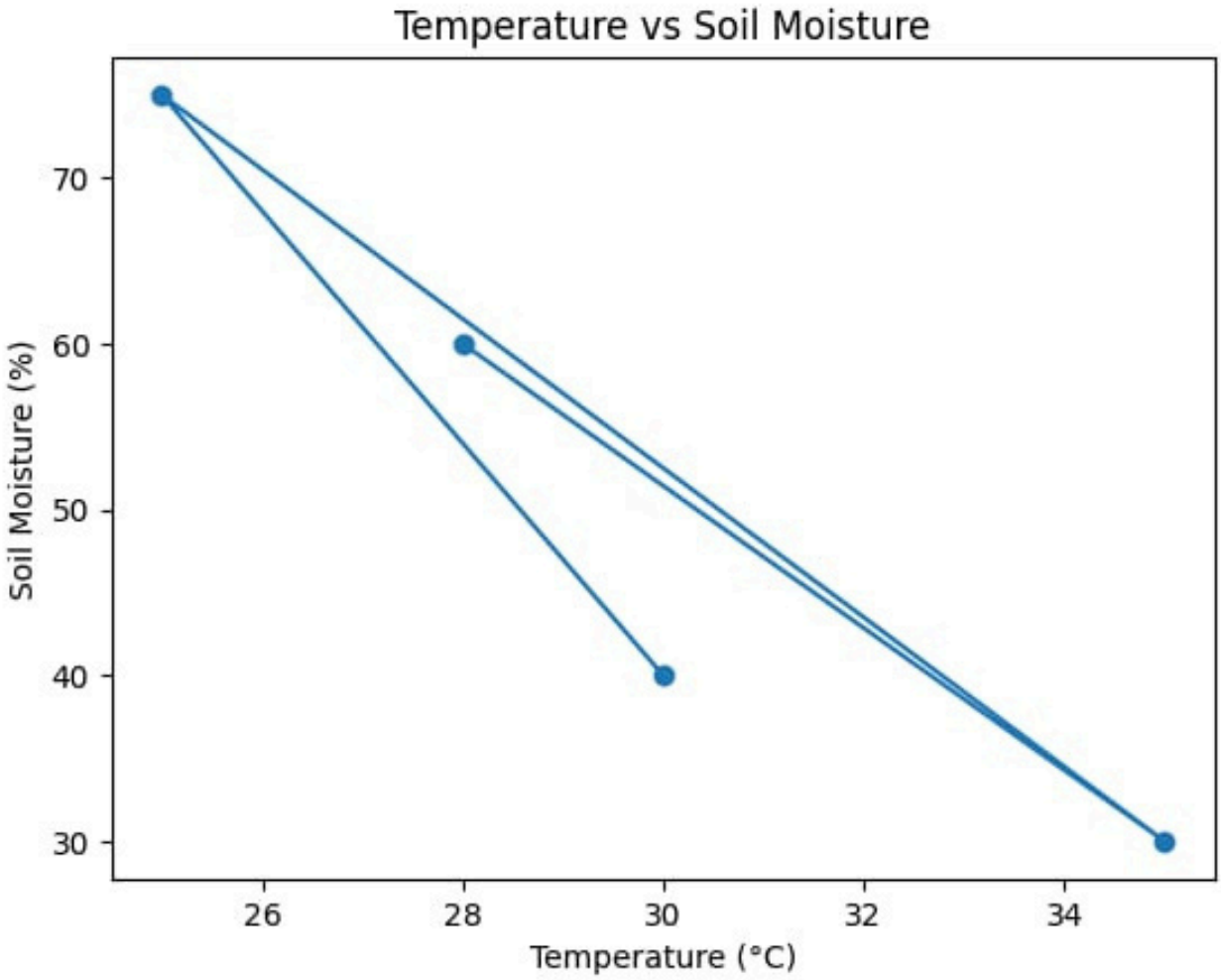
CELL 8 — Graph Visualization

```
temperature = X[:, 0]
humidity = X[:, 1]
soil_moisture = X[:, 2]
rainfall = X[:, 3]

plt.figure(figsize=(10,6))

plt.plot(temperature, label="Temperature",
marker='o')
plt.plot(humidity, label="Humidity", marker='s')
plt.plot(soil_moisture, label="Soil Moisture",
marker='^')
plt.plot(rainfall, label="Rainfall", marker='x')

plt.plot(range(len(y_test)), y_test, label="Actual",
linestyle='--')
plt.plot(range(len(y_pred)), y_pred,
label="Predicted", linestyle='--')

plt.title("Smart Irrigation System using IoT + ML")
plt.xlabel("Data Index")
plt.ylabel("Values")
plt.legend()
```

9.1 Graph Explanation







9.2 How Code Runs (inner content)

Libraries load:

- Loads all required libraries
- Enables ML, graphs, and IoT simulation

IoT Sensor Simulation:

- Creates function to simulate real-time sensor data

Dataset Creation

- X → Input features
- y → Output (water required)
- Model learns:
- Soil dry → more water
- Rainfall high → less water

Train-Test Split

- Splits data:
- 80% training
- 20% testing
- Ensures model is tested properly

Model Training

- Model learns patterns:
  - Rainfall ↑ → irrigation ↓
  - Soil moisture ↓ → irrigation ↑

Evaluation

- Model is tested
- Metrics:
- MAE → error
- R^2 → accuracy
- Cross-validation → stability

Real-Time IoT Prediction

- IoT data generated
- Passed to model
- Prediction shown

9.3 How Code Runs (Final Flow)

1. Libraries load
2. Dataset created
3. Model trained
4. Model evaluated
5. IoT data generated
6. Prediction made
7. Graph displayed

10.DISCUSSION

10.1 Interpretation

- Soil moisture is the most important factor
- Rainfall reduces irrigation requirement
- Model successfully captures real-world patterns

10.2 Strengths

- Real-time IoT simulation
- Accurate prediction
- Easy to implement
- Scalable for smart farming

10.3 Limitations

- Small dataset
- No real hardware integration
- No weather forecast API

11. CONCLUSION

- This project successfully demonstrates the integration of IoT and Machine Learning for smart irrigation. The Random Forest model effectively predicts irrigation requirements, reducing water wastage and improving agricultural efficiency.
- The system is highly suitable for precision agriculture and smart farming applications.

12.FUTURE IMPROVEMENTS

- Real IoT sensors (ESP32, Arduino)
- Mobile app integration
- Weather API integration
- Automatic irrigation control system

13. REFERENCES

1.IoT and Machine Learning-Based Smart Irrigation System

- **Authors:** Rajeev Kumar Srivastava, Sanjeev Kumar Srivastava, Mahima Chaurasia
- **Platform:** IEEE Xplore / Google Scholar
- **Key Idea:** Soil moisture prediction using ML models (RF, SVM)

2.Machine Learning Techniques for Soil Moisture Prediction

- **Authors:** R.K. Srivastava, M. Chaurasia, A. Singh
- **Platform:** ScienceDirect / Google Scholar
- **Key Idea:** Regression-based moisture prediction

3.IoT-Based Smart Agriculture Monitoring System
Authors: R.K. Srivastava, Mahima Chaurasia
Platform: SpringerLink
Key Idea: Real-time sensor monitoring

4.AI-Based Irrigation Control System Using Sensor Data
Authors: R.K. Srivastava, S. Kumar, P. Verma
Platform: IEEE Xplore
Key Idea: Automated irrigation using ML

5.Cloud-Based Smart Irrigation System with Machine Learning

- **Authors:** R.K. Srivastava, Team
- **Platform**: ResearchGate
- **Key Idea:** IoT + cloud irrigation

6.Precision Agriculture Journals

14. APPENDIX

Google Colab
co google.com

:
view google colab code

INPUT VALUES: Using values for prediction:
Temperature: 28.00
Humidity: 18.00
Soil Moisture: 5.00
Rainfall: 0.00
FINAL OUTPUT: Predicted Irrigation Requirement: 58.62

THANKYOU!